

10 ALGORITMOS VISUALES

con

Programación Orientada a Objetos

Clases · Herencia · Encapsulamiento · Composición · localStorage
JavaScript puro · Sin frameworks · Resultados inmediatos en el navegador



Nivel Intermedio



~5 horas práctica



Sin frameworks



10 Proyectos reales

Has llegado al punto de inflexión de tu carrera como desarrollador. Los conceptos que aprenderás en este taller — clases, herencia, encapsulamiento — son la base de React, Vue, Angular, Node.js y prácticamente cualquier framework moderno. Al terminar, pensarás en problemas de software de una manera completamente nueva.

Regla de oro: antes de escribir código, pregúntate "¿qué es esto en el mundo real?" Una clase Producto no es solo código; es la representación digital de un producto con sus características y comportamientos.

#	Ejercicio	Concepto POO	Dificultad
01	Mascota Virtual	Clase, constructor, this, métodos	☆☆
02	Múltiples Mascotas	Múltiples instancias, array de objetos	☆☆
03	Cuenta Bancaria	Encapsulamiento, #private, getters	☆☆☆
04	Figuras Geométricas	extends, super(), polimorfismo	☆☆☆
05	Carrito de Compras	Composición, responsabilidad única	☆☆☆
06	Eventos y Binding	this en callbacks, bind(), arrow fn	☆☆☆
07	Métodos Estáticos	static, utilidades de clase	☆☆☆
08	Notas Persistentes	localStorage, JSON.stringify/parse	☆☆☆☆
09	Galería con Filtros	Clase-componente, estado interno	☆☆☆☆
10	Banco Virtual	Proyecto integrador completo	☆☆☆☆☆



Los 4 Pilares de la POO — Referencia rápida



Encapsulamiento

Los detalles internos están ocultos. Solo accedes por métodos controlados.



Herencia

Una clase hija reutiliza y extiende el código de la clase padre.



Polimorfismo

El mismo método hace cosas distintas según la subclase que lo ejecuta.



Abstracción

Representar lo esencial de un objeto real ignorando detalles irrelevantes.

01



Mi Primera Clase — Mascota Virtual

Ejercicio 01 de 10 · Clase, constructor, propiedades y métodos

Concepto

Definir una clase con constructor() y métodos. Crear una instancia con new. Acceder a propiedades con this.

Metáfora

Una clase es un molde de galletas: el molde no es la galleta, pero con él puedes hacer todas las galletas que quieras, cada una idéntica en forma pero diferente en sabor.



Código HTML completo

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Mascota Virtual</title>
  <style>
    body { font-family: sans-serif; max-width: 480px;
      margin: 40px auto; padding: 20px; background: #F0F4FF; }
    .tarjeta { background: white; border-radius: 16px;
      padding: 24px; box-shadow: 0 4px 16px rgba(0,0,0,.1); }
    .emoji-grande { font-size: 80px; text-align: center; }
    h2 { text-align: center; color: #16213E; }
    .barra-cont { background: #E8EAF6; border-radius: 999px; height: 14px; margin: 6px
0 16px; }
    .barra { height: 14px; border-radius: 999px;
      transition: width .4s ease; }
    .stat-label { display: flex; justify-content: space-between; font-size: 14px;
color: #555; }
    .botones { display: flex; gap: 8px; flex-wrap: wrap; margin-top: 16px; }
    button { flex: 1; padding: 10px; border: none; border-radius: 10px;
      font-size: 15px; cursor: pointer; font-weight: bold; }
    #btn-comer { background: #A8E6CF; color: #1B4332; }
    #btn-jugar { background: #FFD3B6; color: #7D4700; }
    #btn-dormir { background: #D6BCFA; color: #4A148C; }
    #log { margin-top: 14px; font-size: 14px; color: #555;
      min-height: 24px; text-align: center; font-style: italic; }
  </style>
</head>
<body>
  <div class="tarjeta">
    <div class="emoji-grande" id="emoji-mascota">🐾</div>
    <h2 id="nombre-mascota">Cargando...</h2>
    <div class="stat-label"><span>🍖 Hambre</span><span id="val-hambre">0</span></div>
    <div class="barra-cont">
      <div class="barra" id="barra-hambre" style="background:#F87171;width:50%"></div>
    </div>
    <div class="stat-label"><span>⚡ Energía</span><span id="val-
energia">0</span></div>
```

```

    <div class="barra-cont">
      <div class="barra" id="barra-energia" style="background:#34D399;width:80%"></div>
    </div>

    <div class="stat-label"><span>😊 Felicidad</span><span id="val-
felicidad">0</span></div>
    <div class="barra-cont">
      <div class="barra" id="barra-felicidad"
style="background:#60A5FA;width:70%"></div>
    </div>
    <div class="botones">
      <button id="btn-comer" onclick="miMascota.comer()">🍖 Comer</button>
      <button id="btn-jugar" onclick="miMascota.jugar()">🎮 Jugar</button>
      <button id="btn-dormir" onclick="miMascota.dormir()">😴 Dormir</button>
    </div>
    <div id="log"></div>
  </div>

  <script>
    // === TODA LA LÓGICA JS VA AQUÍ ===
  </script>
</body>
</html>

```

JavaScript — Paso a paso

```

// =====
// DEFINICIÓN DE LA CLASE
// Una clase es un plano/molde. Todavía no es un objeto.
// =====

class Mascota {

  // constructor() se ejecuta automáticamente cuando escribes: new Mascota(...)
  // "this" se refiere a ESTA instancia concreta de la clase
  constructor(nombre, emoji) {
    this.nombre = nombre; // propiedad: nombre de la mascota
    this.emoji = emoji; // propiedad: su representación visual
    this.hambre = 50; // 0 = lleno, 100 = muy hambriento
    this.energia = 80; // 0 = agotado, 100 = lleno de energía
    this.felicidad = 70; // 0 = triste, 100 = feliz
  }

  // MÉTODO: comer()
  // Un método es una función que pertenece a la clase
  comer() {
    this.hambre = Math.max(0, this.hambre - 30);
    this.energia = Math.min(100, this.energia + 10);
    this.felicidad = Math.min(100, this.felicidad + 5);
    this.actualizarUI(";Qué rico! 😊 Hambre reducida.");
  }

  jugar() {
    if (this.energia < 20) {

```

```

        this.actualizarUI("Estoy muy cansado para jugar... 🥱");
        return;
    }
    this.energia = Math.max(0, this.energia - 25);
    this.hambre = Math.min(100, this.hambre + 15);
    this.felicidad = Math.min(100, this.felicidad + 20);
    this.actualizarUI(";Woof! Qué divertido 🐾");
}

dormir() {
    this.energia = Math.min(100, this.energia + 40);
    this.felicidad = Math.min(100, this.felicidad + 10);
    this.actualizarUI("Zzzz... 😴 Energía recuperada.");
}

// Este método actualiza el DOM con los valores actuales
actualizarUI(mensaje) {
    document.getElementById("nombre-mascota").innerHTML = this.nombre;
    document.getElementById("emoji-mascota").innerHTML = this.emoji;
    document.getElementById("val-hambre").innerHTML = this.hambre;
    document.getElementById("val-energia").innerHTML = this.energia;
    document.getElementById("val-felicidad").innerHTML = this.felicidad;
    // Las barras se ajustan con el porcentaje como ancho CSS
    document.getElementById("barra-hambre").style.width = this.hambre + "%";
    document.getElementById("barra-energia").style.width = this.energia + "%";
    document.getElementById("barra-felicidad").style.width = this.felicidad + "%";
    document.getElementById("log").innerHTML = mensaje;
}
}

// =====
// CREAR UNA INSTANCIA (un objeto concreto del molde)
// new Mascota() llama al constructor automáticamente
// =====

const miMascota = new Mascota("Fido", "🐶");

// Mostramos el estado inicial en pantalla
miMascota.actualizarUI(";Hola! Soy " + miMascota.nombre + " 🐾");

```



Mini-reto para ti

Agrega una propiedad "edad: 0" y un método "cumpleaños()" que incremente la edad y muestre "¡Feliz cumpleaños! 🎂 Ahora tengo X años."

Agrega una propiedad "nivel: 1" que suba cuando la felicidad llega a 100.

Bonus: Cambia el emoji según el estado de la mascota (triste 😞 si hambre > 80, feliz 😊 si felicidad > 80).

02



Múltiples Mascotas — Arrays de Instancias

Ejercicio 02 de 10 · Múltiples instancias, array de objetos, selector dinámico

Concepto

Crear varias instancias de la misma clase, almacenarlas en un array y acceder a la instancia correcta usando un índice.

Metáfora

Si la clase es el molde, las instancias son cada galleta individual. Puedes hacer 10 galletas con el mismo molde y decorar cada una distinto.

Código HTML completo

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8"><title>Mis Mascotas</title>
  <style>
    body { font-family:sans-serif; max-width:600px; margin:30px auto; padding:20px;
background:#F0FBF1; }
    .selector { display:flex; gap:10px; flex-wrap:wrap; margin-bottom:20px; }
    .btn-mascota { padding:12px 20px; border:3px solid transparent;
border-radius:12px; font-size:22px; cursor:pointer;
background:white; transition:all .2s; }
    .btn-mascota.activa { border-color:#2D6A4F; background:#D8F3DC; }
    .tarjeta { background:white; border-radius:16px; padding:24px;
box-shadow:0 4px 16px rgba(0,0,0,.1); }
    .stats { display:grid; grid-template-columns:1fr 1fr 1fr; gap:12px; margin:16px 0;
}
    .stat { text-align:center; background:#F0FBF1; padding:12px;
border-radius:10px; font-size:14px; }
    .stat.num { font-size:28px; font-weight:bold; color:#2D6A4F; }
    .acciones { display:flex; gap:8px; margin-top:14px; }
    .acciones button { flex:1; padding:10px; border:none; border-radius:10px;
cursor:pointer; font-size:15px; font-weight:bold;
background:#D8F3DC; color:#1B4332; }
  </style>
</head>
<body>
  <h2>🐾 Mis Mascotas</h2>
  <div class="selector" id="selector"></div>
  <div class="tarjeta">
    <div id="nombre-actual" style="font-size:22px;font-
weight:bold;color:#1B4332"></div>
    <div class="stats">
      <div class="stat"><div class="num" id="s-hambre"></div>🍌 Hambre</div>
      <div class="stat"><div class="num" id="s-energia"></div>⚡ Energía</div>
      <div class="stat"><div class="num" id="s-felicidad"></div>😊 Felicidad</div>
    </div>
    <div class="acciones">
      <button onclick="accion('comer')">🍌 Comer</button>
      <button onclick="accion('jugar')">🎮 Jugar</button>
    </div>
  </div>
</body>
</html>
```

```

        <button onclick="accion('dormir')">😴 Dormir</button>
    </div>
    <p id="log-2" style="text-align:center;color:#555;font-style:italic;margin-top:10px;"></p>
</div>
<script>/* código JS aquí */</script>
</body></html>

```



JavaScript — Paso a paso

```

class Mascota {
    constructor(nombre, emoji) {
        this.nombre = nombre;
        this.emoji = emoji;
        this.hambre = Math.floor(Math.random() * 60) + 20; // valor inicial aleatorio
        this.energia = Math.floor(Math.random() * 60) + 30;
        this.felicidad = Math.floor(Math.random() * 60) + 40;
    }
    comer() { this.hambre = Math.max(0, this.hambre-30); this.energia = Math.min(100,this.energia+10); }
    jugar() { this.energia = Math.max(0,this.energia-25); this.felicidad = Math.min(100,this.felicidad+20); }
    dormir() { this.energia = Math.min(100,this.energia+40); }
}

// =====
// ARRAY DE INSTANCIAS: varias mascotas, cada una
// es un objeto independiente en memoria
// =====
const mascotas = [
    new Mascota("Fido", "🐶"),
    new Mascota("Michi", "🐱"),
    new Mascota("Tweety", "🐤"),
    new Mascota("Nemo", "🐠"),
];

// Índice de la mascota actualmente seleccionada
let indiceActual = 0;

function construirSelector() {
    const cont = document.getElementById("selector");
    cont.innerHTML = "";
    for (let i = 0; i < mascotas.length; i++) {
        // Marcamos el botón como activo si es el índice actual
        const clase = i === indiceActual ? "btn-mascota activa" : "btn-mascota";
        cont.innerHTML +=
            "<button class=\"\" + clase + \"\" onclick=\"seleccionar(" + i + ")\">\" +
            mascotas[i].emoji + \" \" + mascotas[i].nombre + "</button>";
    }
}

function seleccionar(i) {

```

```

    indiceActual = i; // guardamos cuál está activa
    construirSelector();
    renderMascota();
}

function renderMascota() {
    // Accedemos a la instancia por su índice en el array
    const m = mascotas[indiceActual];
    document.getElementById("nombre-actual").innerHTML = m.emoji + " " + m.nombre;
    document.getElementById("s-hambre").innerHTML = m.hambre;
    document.getElementById("s-energia").innerHTML = m.energia;
    document.getElementById("s-felicidad").innerHTML = m.felicidad;
}

function accion(tipo) {
    const m = mascotas[indiceActual];
    if (tipo === "comer") m.comer();
    if (tipo === "jugar") m.jugar();
    if (tipo === "dormir") m.dormir();
    document.getElementById("log-2").innerHTML =
        m.nombre + " hizo: " + tipo + " ✅";
    renderMascota();
}

// Inicializar
construirSelector();
renderMascota();

```



Mini-reto para ti

Agrega una mascota más al array con tu animal favorito.

Muestra en la tarjeta "La más feliz es: [nombre]" buscando con un for la que tenga mayor felicidad.

Bonus: Agrega un botón "Alimentar a todas" que llame a .comer() de cada mascota usando un for.

03



Encapsulamiento — Cuenta Bancaria

Ejercicio 03 de 10 · Propiedades "privadas" (#), getters, setters y validación

Concepto

Usar # para declarar propiedades privadas que no pueden tocarse desde fuera de la clase. Exponer acceso controlado con get/set o métodos específicos.

Metáfora

El saldo real de tu cuenta bancaria es privado. No puedes modificarlo directamente desde afuera: solo el banco (la clase) puede hacerlo, y únicamente a través de operaciones validadas (depositar, retirar).

Código HTML completo

```
<!DOCTYPE html>
<html lang="es"><head>
  <meta charset="UTF-8"><title>Cuenta Bancaria</title>
  <style>
    body { font-family:sans-serif; max-width:480px; margin:40px auto;
      padding:20px; background:#FFF5F5; }
    .banco { background:white; border-radius:16px; padding:24px;
      box-shadow:0 4px 20px rgba(192,57,43,.15); }
    .saldo { text-align:center; font-size:52px; font-weight:bold;
      color:#B03A2E; margin:8px 0; }
    .titular { text-align:center; color:#888; font-size:15px; margin-bottom:20px; }
    .fila { display:flex; gap:8px; margin-bottom:10px; }
    input { flex:1; padding:10px; font-size:16px; border:2px solid #FADADD;
      border-radius:8px; }
    .btn-dep { padding:10px 18px; background:#A9DFBF; color:#1B4332;
      border:none; border-radius:8px; font-weight:bold;
      font-size:15px; cursor:pointer; }
    .btn-ret { padding:10px 18px; background:#FADADD; color:#7D2200;
      border:none; border-radius:8px; font-weight:bold;
      font-size:15px; cursor:pointer; }
    #historial { margin-top:16px; max-height:160px; overflow-y:auto; }
    .mov { font-size:14px; padding:6px 10px; border-radius:6px; margin-bottom:4px; }
    .entrada { background:#F0FDF4; color:#166534; }
    .salida { background:#FEE2F2; color:#991B1B; }
    .error { color:#B91C1C; font-size:14px; text-align:center; margin-top:8px; }
  </style>
</head><body>
  <div class="banco">
    <div class="titular"><img alt="person icon" data-bbox="318 765 335 778"/> Cuenta de: <strong id="titular"></strong></div>
    <div class="saldo"><span id="saldo-display">0.00</span></div>
    <div class="fila">
      <input id="monto" type="number" placeholder="Monto" min="0">
      <button class="btn-dep" onclick="depositar()"><img alt="up arrow icon" data-bbox="545 825 560 838"/> Depositar</button>
      <button class="btn-ret" onclick="retirar()"><img alt="down arrow icon" data-bbox="525 840 540 853"/> Retirar</button>
    </div>
    <p id="error-msg" class="error"></p>
    <div id="historial"></div>
  </div>
```

```
<script>/* código JS */</script>
</body></html>
```

JavaScript — Paso a paso

```
class CuentaBancaria {

  // # hace que la propiedad sea PRIVADA (solo accesible dentro de la clase)
  // No se puede leer ni modificar desde fuera: cuenta.#saldo → ERROR
  #saldo = 0;
  #historial = [];

  constructor(titular, saldoInicial = 0) {
    this.titular = titular;
    if (saldoInicial > 0) {
      this.#saldo = saldoInicial;
      this.#historial.push({ tipo:"apertura", monto:saldoInicial });
    }
  }

  // GETTER: permite LEER el saldo como si fuera una propiedad normal
  // cuenta.saldo → llama a esta función automáticamente
  get saldo() {
    return this.#saldo;
  }

  // MÉTODO depositar: valida antes de modificar el saldo privado
  depositar(monto) {
    if (monto <= 0) throw new Error("El monto debe ser mayor a 0");
    this.#saldo += monto;
    this.#historial.push({ tipo:"deposito", monto });
    return this; // permite encadenar: cuenta.depositar(100).depositar(50)
  }

  retirar(monto) {
    if (monto <= 0)          throw new Error("El monto debe ser mayor a 0");
    if (monto > this.#saldo)  throw new Error("Saldo insuficiente");
    this.#saldo -= monto;
    this.#historial.push({ tipo:"retiro", monto });
    return this;
  }

  getHistorial() {
    // Devolvemos una COPIA para proteger el array interno
    return [...this.#historial];
  }
}

// Crear la cuenta
const cuenta = new CuentaBancaria("Ana García", 500);
```

```

function render() {
  document.getElementById("titular").innerHTML = cuenta.titular;
  // Usamos el getter: cuenta.saldo (no cuenta.#saldo)
  document.getElementById("saldo-display").innerHTML = cuenta.saldo.toFixed(2);
  const hist = document.getElementById("historial");
  hist.innerHTML = "";
  const movimientos = cuenta.getHistorial().reverse();
  for (let i = 0; i < movimientos.length; i++) {
    const m = movimientos[i];
    const clase = m.tipo === "retiro" ? "mov salida" : "mov entrada";
    const signo = m.tipo === "retiro" ? "-" : "+";
    hist.innerHTML += "<div class=\"" + clase + "\">",
      + signo + " $" + m.monto.toFixed(2)
      + " <em>(" + m.tipo + ")</em></div>";
  }
}

function depositar() {
  const monto = Number(document.getElementById("monto").value);
  const err = document.getElementById("error-msg");
  try {
    cuenta.depositar(monto);
    err.innerHTML = "";
    render();
  } catch(e) {
    err.innerHTML = "⚠ " + e.message;
  }
}

function retirar() {
  const monto = Number(document.getElementById("monto").value);
  const err = document.getElementById("error-msg");
  try {
    cuenta.retirar(monto);
    err.innerHTML = "";
    render();
  } catch(e) {
    err.innerHTML = "⚠ " + e.message;
  }
}

render();

```

Mini-reto para ti

Intenta acceder directamente a `cuenta.#saldo` desde la consola. ¿Qué error aparece?

Agrega un método `transferir(destino, monto)` que retire de esta cuenta y deposite en otra instancia.

Bonus: Agrega un getter `"resumen"` que devuelva un string: `"Titular: X | Saldo: $Y | Movimientos: Z"`.

04



Herencia — Figuras Geométricas

Ejercicio 04 de 10 · extends, super(), polimorfismo, método override

Concepto

Una clase hija hereda propiedades y métodos de la clase padre. `super()` llama al constructor del padre. El polimorfismo permite que cada subclase tenga su propia versión de un método.

Metáfora

Figura es como "vehículo": define que todo vehículo tiene velocidad y color. Círculo y Rectángulo son como "auto" y "moto": heredan esas bases pero tienen sus propias formas de calcular su área.



Código HTML completo

```
<!DOCTYPE html>
<html lang="es"><head>
  <meta charset="UTF-8"><title>Figuras Geométricas</title>
  <style>
    body { font-family:sans-serif; max-width:560px; margin:30px auto;
      padding:20px; background:#F5EEFF; }
    .controles { background:white; border-radius:14px; padding:20px;
      margin-bottom:16px; box-shadow:0 2px 12px rgba(0,0,0,.08); }
    select,input { padding:8px 12px; font-size:15px; border:2px solid #D8B4FE;
      border-radius:8px; margin:4px 0; }
    button { padding:10px 20px; background:#8E24AA; color:white; border:none;
      border-radius:8px; font-size:15px; cursor:pointer; margin-top:8px; }
    #canvas-cont { background:white; border-radius:14px; padding:24px;
      text-align:center; min-height:200px;
      box-shadow:0 2px 12px rgba(0,0,0,.08); }
    canvas { border:2px dashed #D8B4FE; border-radius:8px; }
    .info-figura { margin-top:14px; font-size:16px; color:#4A148C; font-weight:bold; }
  </style>
</head><body>
  <h2>◆ Figuras Geométricas</h2>
  <div class="controles">
    <label>Tipo: <select id="tipo">
      <option value="circulo">Círculo</option>
      <option value="rectangulo">Rectángulo</option>
      <option value="triangulo">Triángulo</option>
    </select></label><br>
    <label>Valor 1 (radio / ancho / base): <input id="v1" type="number" value="80"
    min="10" max="150"></label><br>
    <label>Valor 2 (- / alto / altura): <input id="v2" type="number" value="60"
    min="10" max="150"></label><br>
    <input id="color-fig" type="color" value="#8E24AA">
    <button onclick="dibujar()">▶ Dibujar</button>
  </div>
  <div id="canvas-cont">
    <canvas id="lienzo" width="400" height="220"></canvas>
    <div class="info-figura" id="info-figura"></div>
  </div>
```

```
<script>/* código JS */</script>
</body></html>
```

JavaScript — Paso a paso

```
// =====
// CLASE BASE (padre) - define lo que TODA figura tiene
// =====
class Figura {
  constructor(color) {
    this.color = color;
  }

  // Método que TODAS las figuras deben tener (pero cada una lo implementa distinto)
  area() {
    return 0; // valor por defecto en la clase base
  }

  descripcion() {
    return "Figura con área: " + this.area().toFixed(2);
  }
}

// =====
// CLASES HIJAS: "extends" indica herencia
// "super()" llama al constructor del padre
// =====
class Circulo extends Figura {
  constructor(radius, color) {
    super(color); // llama a Figura.constructor(color)
    this.radius = radius; // propiedad EXCLUSIVA del círculo
  }

  // Override: nuestra propia versión del método area()
  area() {
    return Math.PI * this.radius * this.radius;
  }

  dibujarEn(ctx, cx, cy) {
    ctx.beginPath();
    ctx.arc(cx, cy, this.radius, 0, Math.PI * 2);
    ctx.fillStyle = this.color;
    ctx.fill();
  }
}

class Rectangulo extends Figura {
  constructor(ancho, alto, color) {
    super(color);
    this.ancho = ancho;
    this.alto = alto;
  }
}
```

```

    area() { return this.ancho * this.alto; }

    dibujarEn(ctx, cx, cy) {
        ctx.fillStyle = this.color;
        ctx.fillRect(cx - this.ancho/2, cy - this.alto/2, this.ancho, this.alto);
    }
}

class Triangulo extends Figura {
    constructor(base, altura, color) {
        super(color);
        this.base = base;
        this.altura = altura;
    }

    area() { return (this.base * this.altura) / 2; }

    dibujarEn(ctx, cx, cy) {
        ctx.fillStyle = this.color;
        ctx.beginPath();
        ctx.moveTo(cx, cy - this.altura/2);
        ctx.lineTo(cx + this.base/2, cy + this.altura/2);
        ctx.lineTo(cx - this.base/2, cy + this.altura/2);
        ctx.closePath();
        ctx.fill();
    }
}

function dibujar() {
    const tipo = document.getElementById("tipo").value;
    const v1 = Number(document.getElementById("v1").value);
    const v2 = Number(document.getElementById("v2").value);
    const color = document.getElementById("color-fig").value;

    // Polimorfismo: figura apunta a cualquier tipo, pero cada uno dibuja y calcula
    // diferente
    let figura;
    if (tipo === "circulo") figura = new Circulo(v1, color);
    else if (tipo === "rectangulo") figura = new Rectangulo(v1, v2, color);
    else figura = new Triangulo(v1, v2, color);

    const canvas = document.getElementById("lienzo");
    const ctx = canvas.getContext("2d");
    ctx.clearRect(0, 0, canvas.width, canvas.height);
    figura.dibujarEn(ctx, 200, 110);

    // descripcion() llama a area() que cada clase implementó distinto
    document.getElementById("info-figura").innerHTML = figura.descripcion();
}

dibujar();

```



Mini-reto para ti

Crea una clase Rombo que extienda Figura. Su área es: $(\text{diagonal1} * \text{diagonal2}) / 2$.

Modifica `descripcion()` en Circulo para que también muestre el perímetro ($2 * \pi * \text{radio}$).

Bonus: Agrega un botón "Dibujar todas" que cree una instancia de cada tipo y las dibuje en posiciones distintas del canvas.

05



Composición — Carrito de Compras

Ejercicio 05 de 10 · Objetos dentro de objetos, responsabilidad única por clase

Concepto

La composición combina objetos simples para construir estructuras complejas. Un Carrito contiene Productos. Cada clase tiene una responsabilidad única y clara.

Metáfora

Es como un equipo de trabajo: el Producto sabe cuánto cuesta y tiene nombre, pero no sabe nada del carrito. El Carrito sabe cómo gestionar su lista de productos, pero no sabe cómo calcular precios individuales.

Código HTML completo

```
<!DOCTYPE html>
<html lang="es"><head>
  <meta charset="UTF-8"><title>Carrito de Compras</title>
  <style>
    body { font-family:sans-serif; max-width:580px; margin:30px auto;
      padding:20px; background:#FFFDF0; }
    .panel { background:white; border-radius:14px; padding:20px;
      box-shadow:0 2px 12px rgba(0,0,0,.08); margin-bottom:16px; }
    .catalogo { display:grid; grid-template-columns:repeat(3,1fr); gap:10px; }
    .prod-btn { padding:14px 8px; background:#FCF3CF; border:2px solid #F9E79F;
      border-radius:10px; cursor:pointer; font-size:14px;
      text-align:center; transition:transform .15s; }
    .prod-btn:hover { transform:scale(1.06); }
    .prod-btn .precio { font-weight:bold; color:#D4830A; }
    #lista-carrito { min-height:60px; }
    .item-carrito { display:flex; justify-content:space-between; align-items:center;
      padding:10px 14px; background:#FFF8E1; border-radius:8px;
      margin-bottom:6px; }
    .quitar { background:#FADADD; border:none; border-radius:6px;
      padding:4px 10px; cursor:pointer; color:#7D2200; font-size:13px; }
    .total-box { background:#FCF3CF; border-radius:10px; padding:14px 18px;
      font-size:20px; font-weight:bold; color:#D4830A; text-align:center; }
  </style>
</head><body>
  <h2>&img alt="Shopping cart icon" data-bbox="161 708 178 719"/> Carrito de Compras</h2>
  <div class="panel">
    <strong>&img alt="Shopping cart icon" data-bbox="211 738 228 749"/> Catálogo</strong>
    <div class="catalogo" id="catalogo"></div>
  </div>
  <div class="panel">
    <strong>&img alt="Shopping cart icon" data-bbox="211 803 228 814"/> Carrito (<span id="cant-items">0</span> items)</strong>
    <div id="lista-carrito"></div>
    <div class="total-box" id="total-box">Total: $0.00</div>
    <button onclick="vaciar()" style="margin-top:10px;padding:8px 16px;
      background:#FADADD;border:none;border-radius:8px;cursor:pointer;">
      &img alt="Shopping cart icon" data-bbox="161 873 178 884"/> Vaciar carrito</button>
  </div>
<script>/* código JS */</script>
```


</body></html>

JavaScript — Paso a paso

```
// CLASE 1: Producto - sabe su nombre, precio y emoji
class Producto {
  constructor(nombre, precio, emoji) {
    this.nombre = nombre;
    this.precio = precio;
    this.emoji = emoji;
  }

  // Describe el producto como string
  toString() {
    return this.emoji + " " + this.nombre + " - $" + this.precio.toFixed(2);
  }
}

// CLASE 2: Carrito - contiene una lista de instancias de Producto
class Carrito {
  constructor() {
    // Composición: el carrito CONTIENE un array de objetos Producto
    this.items = [];
  }

  agregar(producto) {
    // Buscamos si ya existe para incrementar cantidad
    for (let i = 0; i < this.items.length; i++) {
      if (this.items[i].producto.nombre === producto.nombre) {
        this.items[i].cantidad++;
        return;
      }
    }
    // Si no existe, lo añadimos como nuevo ítem
    this.items.push({ producto, cantidad: 1 });
  }

  quitar(indice) {
    if (this.items[indice].cantidad > 1) {
      this.items[indice].cantidad--;
    } else {
      this.items.splice(indice, 1);
    }
  }

  calcularTotal() {
    let total = 0;
    for (let i = 0; i < this.items.length; i++) {
      total += this.items[i].producto.precio * this.items[i].cantidad;
    }
    return total;
  }
}
```

```

    vaciar() { this.items = []; }
}

// Catálogo de productos disponibles
const catalogo = [
    new Producto("Manzana", 1.20, "🍏"),
    new Producto("Banana", 0.80, "🍌"),
    new Producto("Leche", 2.50, "🥛"),
    new Producto("Pan", 1.80, "🍞"),
    new Producto("Queso", 4.30, "🧀"),
    new Producto("Café", 3.60, "☕"),
];

const carrito = new Carrito();

function renderCatalogo() {
    const cont = document.getElementById("catalogo");
    cont.innerHTML = "";
    for (let i = 0; i < catalogo.length; i++) {
        const p = catalogo[i];
        cont.innerHTML +=
            "<div class=\"prod-btn\" onclick=\"agregar(" + i + ")\">" +
            " <div style=\"font-size:30px;\"> " + p.emoji + "</div>" +
            " <div>" + p.nombre + "</div>" +
            " <div class=\"precio\">$" + p.precio.toFixed(2) + "</div>" +
            "</div>";
    }
}

function agregar(i) {
    carrito.agregar(catalogo[i]);
    renderCarrito();
}

function quitarItem(i) {
    carrito.quitar(i);
    renderCarrito();
}

function vaciar() {
    carrito.vaciar();
    renderCarrito();
}

function renderCarrito() {
    const lista = document.getElementById("lista-carrito");
    lista.innerHTML = "";
    let totalItems = 0;
    for (let i = 0; i < carrito.items.length; i++) {
        const it = carrito.items[i];
        totalItems += it.cantidad;
        const sub = it.producto.precio * it.cantidad;
        lista.innerHTML +=

```

```
"<div class=\"item-carrito\">" +  
"  <span>" + it.producto.emoji + " " + it.producto.nombre +  
    " x" + it.cantidad + "</span>" +  
"  <span style=\"font-weight:bold;\">$" + sub.toFixed(2) + "</span>" +  
"  <button class=\"quitar\" onclick=\"quitarItem(" + i + ")\">-</button>" +  
"</div>";  
  
}  
document.getElementById("cant-items").innerHTML = totalItems;  
document.getElementById("total-box").innerHTML =  
  "Total: $" + carrito.calcularTotal().toFixed(2);  
}  
  
renderCatalogo();  
renderCarrito();
```



Mini-reto para ti

Agrega un campo "stock" al Producto. Si el stock llega a 0, deshabilita el botón del catálogo.

Agrega un método aplicarDescuento(porcentaje) al Carrito que reduzca el total.

Bonus: Muestra cuánto se ahorra si aplicas un 10% de descuento en un elemento destacado del panel.

06



Eventos y Binding — El problema del this

Ejercicio 06 de 10 · `addEventListener` dentro de clases, arrow functions, `bind()`

Concepto

Cuando usas `addEventListener` dentro de un método de clase, `"this"` deja de apuntar a la instancia. Las soluciones son: arrow functions, `.bind(this)`, o guardar la referencia en una variable.

Metáfora

`"this"` es como una etiqueta de "remitente" en una carta. Si alguien más reenvía la carta (el event listener), la etiqueta del remitente cambia. Necesitas fijarla explícitamente.



Código HTML completo

```
<!DOCTYPE html>
<html lang="es"><head>
  <meta charset="UTF-8"><title>Eventos y Binding</title>
  <style>
    body { font-family:sans-serif; max-width:560px; margin:30px auto;
      padding:20px; background:#E0F7FA; }
    .demo-box { background:white; border-radius:14px; padding:22px;
      box-shadow:0 2px 12px rgba(0,0,0,.1); margin-bottom:16px; }
    h3 { color:#006064; margin-top:0; }
    .cont-clics { display:flex; align-items:center; gap:14px; }
    .num-clics { font-size:52px; font-weight:bold; color:#00ACC1; }
    button { padding:12px 24px; background:#00838F; color:white; border:none;
      border-radius:10px; font-size:15px; cursor:pointer; }
    .historial-item { background:#E0F7FA; padding:8px 12px; border-radius:6px;
      margin-bottom:5px; font-size:14px; }
    .tecla { display:inline-block; background:#B2EBF2; padding:3px 8px;
      border-radius:4px; font-family:monospace; color:#006064; }
  </style>
</head><body>
  <h2>🔗 Eventos dentro de Clases</h2>
  <div class="demo-box">
    <h3>Contador de Clics</h3>
    <div class="cont-clics">
      <div class="num-clics" id="num-clics">0</div>
      <div>
        <button id="btn-click">Haz clic!</button>
        <p style="font-size:13px;color:#888">O presiona la tecla <span
class="tecla">Espacio</span></p>
      </div>
    </div>
  </div>
  <div class="demo-box">
    <h3>📋 Historial de eventos</h3>
    <div id="historial"></div>
  </div>
  <script>/* código JS */</script>
</body></html>
```

```
class Contador {
  constructor(id) {
    this.clics = 0;
    this.historial = [];
    this.elementoId = id;

    // =====
    // PROBLEMA: si hacemos esto, "this" en manejarClic
    // apuntaría al botón (el elemento que disparó el evento)
    // y NO a la instancia de Contador
    // ❌ btn.addEventListener("click", this.manejarClic);
    // =====

    // SOLUCIÓN 1: Arrow function anónima (la más común)
    // La arrow function hereda el "this" del contexto exterior
    const btn = document.getElementById("btn-click");
    btn.addEventListener("click", () => {
      this.manejarClic("clic en botón"); // "this" correcto ✅
    });

    // SOLUCIÓN 2: .bind(this) - fija "this" explícitamente
    document.addEventListener("keydown", this.manejarTecla.bind(this));
  }

  manejarClic(fuente) {
    this.clics++;
    const hora = new Date().toLocaleTimeString();
    this.historial.unshift({ fuente, hora, num: this.clics });
    // Limitar historial a 6 entradas
    if (this.historial.length > 6) this.historial.pop();
    this.render();
  }

  manejarTecla(event) {
    if (event.code === "Space") {
      event.preventDefault();
      // "this" apunta a la instancia gracias a .bind(this)
      this.manejarClic("tecla Espacio");
    }
  }

  render() {
    document.getElementById(this.elementoId).innerHTML = this.clics;
    const cont = document.getElementById("historial");
    cont.innerHTML = "";
    for (let i = 0; i < this.historial.length; i++) {
      const h = this.historial[i];
      cont.innerHTML +=
        "<div class=\"historial-item\">" +
        "  #" + h.num + "  ·  " + h.fuente + "  ·  " + h.hora +

```

```
        "</div>";
    }
}

// Crear la instancia pasando el id del elemento del DOM
const contador = new Contador("num-clics");
```



Mini-reto para ti

Agrega un botón "Resetear" que ponga clics = 0 e historial = [].

Agrega soporte para la tecla "Enter": también cuenta como clic.

Bonus: Guarda en this.record el máximo de clics en 5 segundos. Usa setTimeout y setInterval dentro del constructor.

07



Métodos Estáticos — Utilidades de Clase

Ejercicio 07 de 10 · static, métodos de clase vs métodos de instancia

Concepto

Un método static pertenece a la CLASE, no a una instancia. Se llama con NombreClase.metodo() sin necesidad de crear un objeto con new.

Metáfora

Math.round() es un método estático: lo llamas directamente en Math, sin crear "un objeto Math". Es una herramienta de la clase, no de cada objeto que creas con ella.



Código HTML completo

```
<!DOCTYPE html>
<html lang="es"><head>
  <meta charset="UTF-8"><title>Métodos Estáticos</title>
  <style>
    body { font-family:sans-serif; max-width:560px; margin:30px auto;
      padding:20px; background:#E8EAF6; }
    .panel { background:white; border-radius:14px; padding:20px;
      box-shadow:0 2px 12px rgba(0,0,0,.08); margin-bottom:14px; }
    h3 { color:#283593; margin-top:0; }
    .grid-2 { display:grid; grid-template-columns:1fr 1fr; gap:10px; }
    input,select { padding:9px; font-size:15px; border:2px solid #C5CAE9;
      border-radius:8px; width:100%; box-sizing:border-box; }
    button { width:100%; margin-top:10px; padding:11px; background:#3949AB;
      color:white; border:none; border-radius:8px;
      font-size:15px; cursor:pointer; }
    .resultado { background:#E8EAF6; border-radius:8px; padding:14px;
      margin-top:10px; font-size:18px; color:#1A237E;
      font-weight:bold; text-align:center; }
    .estadistica { display:flex; justify-content:space-between; padding:6px 0;
      border-bottom:1px solid #E8EAF6; font-size:15px; }
  </style>
</head><body>
  <h2>📊 Calculadora de Estadísticas</h2>
  <div class="panel">
    <h3>🔢 Convertidor de unidades</h3>
    <div class="grid-2">
      <input id="valor-conv" type="number" placeholder="Valor">
      <select id="tipo-conv">
        <option value="kmMillas">km → millas</option>
        <option value="millaKm">millas → km</option>
        <option value="cF">°C → °F</option>
        <option value="fC">°F → °C</option>
      </select>
    </div>
    <button onclick="convertir()">Convertir</button>
    <div class="resultado" id="res-conv"></div>
  </div>
  <div class="panel">
    <h3>🇺🇸 Estadísticas de una lista</h3>
```

```

    <input id="lista-nums" type="text" placeholder="Números separados por comas:
10,20,30">
    <button onclick="calcularStats()">Calcular</button>
    <div id="res-stats"></div>
</div>
<script>/* código JS */</script>
</body></html>

```



JavaScript — Paso a paso

```

// =====
// Clase con SOLO métodos estáticos: es una caja de herramientas.
// Nunca necesitamos hacer: new Calculadora()
// =====
class Calculadora {

    // static: se llama como Calculadora.kmAMillas(5)
    static kmAMillas(km)    { return km * 0.621371; }
    static millasAKm(millas) { return millas * 1.60934; }
    static celsiusAFahr(c)   { return (c * 9/5) + 32; }
    static fahrACelsius(f)   { return (f - 32) * 5/9; }

    static convertir(valor, tipo) {
        if (tipo === "kmMillas") return Calculadora.kmAMillas(valor);
        if (tipo === "millaKm")  return Calculadora.millasAKm(valor);
        if (tipo === "cF")        return Calculadora.celsiusAFahr(valor);
        if (tipo === "fC")        return Calculadora.fahrACelsius(valor);
    }
}

// =====
// Otra clase de utilidades: estadísticas de arrays
// =====
class Estadisticas {

    static suma(nums) {
        let total = 0;
        for (let i = 0; i < nums.length; i++) total += nums[i];
        return total;
    }

    static promedio(nums) {
        return Estadisticas.suma(nums) / nums.length;
    }

    static maximo(nums) {
        let max = nums[0];
        for (let i = 1; i < nums.length; i++) if (nums[i] > max) max = nums[i];
        return max;
    }

    static minimo(nums) {

```



```

    let min = nums[0];
    for (let i = 1; i < nums.length; i++) if (nums[i] < min) min = nums[i];
    return min;
}

// Desviación estándar: mide cuánto se alejan los datos del promedio
static desviacion(nums) {
    const prom = Estadisticas.promedio(nums);
    let sumDif = 0;
    for (let i = 0; i < nums.length; i++) sumDif += (nums[i] - prom) ** 2;
    return Math.sqrt(sumDif / nums.length);
}
}

function convertir() {
    const valor = Number(document.getElementById("valor-conv").value);
    const tipo = document.getElementById("tipo-conv").value;
    const res = Calculadora.convertir(valor, tipo);
    document.getElementById("res-conv").innerHTML = res.toFixed(4);
}

function calcularStats() {
    const texto = document.getElementById("lista-nums").value;
    const nums = texto.split(",").map(n => Number(n.trim())).filter(n => !isNaN(n));
    if (nums.length === 0) return;

    const stats = [
        ["Suma", Estadisticas.suma(nums).toFixed(2)],
        ["Promedio", Estadisticas.promedio(nums).toFixed(2)],
        ["Máximo", Estadisticas.maximo(nums)],
        ["Mínimo", Estadisticas.minimo(nums)],
        ["Desviación", Estadisticas.desviacion(nums).toFixed(2)],
    ];

    let html = "";
    for (let i = 0; i < stats.length; i++) {
        html += "<div class=\"estadistica\"><span>" + stats[i][0] +
            "</span><strong>" + stats[i][1] + "</strong></div>";
    }
    document.getElementById("res-stats").innerHTML = html;
}

```



Mini-reto para ti

Agrega un método estático `Estadisticas.mediana(nums)` que ordene el array y devuelva el valor central.

Agrega un contador estático `Calculadora.totalConversiones = 0` que suba cada vez que se llama a `convertir()`.

Bonus: Crea una clase `Validador` con métodos estáticos: `esEmail(str)`, `esNumeroPositivo(n)`, `tieneLongitud(str, min, max)`.

Persistencia con localStorage

Ejercicio 08 de 10 · JSON.stringify, JSON.parse, guardar/recuperar objetos

 Concepto	localStorage guarda strings en el navegador de forma permanente. Para guardar objetos usamos <code>JSON.stringify()</code> (objeto → texto) y para leerlos <code>JSON.parse()</code> (texto → objeto).
 Metáfora	localStorage es como un cuaderno que queda sobre el escritorio aunque apagues el computador. <code>JSON.stringify</code> es escribir en el cuaderno; <code>JSON.parse</code> es volver a leer lo escrito.

 Código HTML completo

```
<!DOCTYPE html>
<html lang="es"><head>
  <meta charset="UTF-8"><title>Notas Persistentes</title>
  <style>
    body { font-family:sans-serif; max-width:560px; margin:30px auto;
      padding:20px; background:#EEF6FD; }
    .panel { background:white; border-radius:14px; padding:20px;
      box-shadow:0 2px 12px rgba(0,0,0,.08); margin-bottom:14px; }
    textarea { width:100%; height:80px; padding:10px; font-size:15px;
      border:2px solid #BFDDBFE; border-radius:8px;
      box-sizing:border-box; resize:vertical; }
    .botones { display:flex; gap:8px; margin-top:8px; }
    button { flex:1; padding:10px; border:none; border-radius:8px;
      font-size:14px; cursor:pointer; font-weight:bold; }
    .btn-guardar { background:#BFDDBFE; color:#1E40AF; }
    .btn-limpiar { background:#FECACA; color:#991B1B; }
    .nota-card { background:#FFF6FF; border-left:4px solid #1B6CA8;
      padding:12px; border-radius:0 8px 8px 0;
      margin-bottom:8px; position:relative; }
    .nota-fecha { font-size:11px; color:#6B7280; margin-bottom:4px; }
    .nota-texto { font-size:15px; }
    .btn-del-nota { position:absolute; top:8px; right:8px;
      background:#FEE2E2; border:none; border-radius:6px;
      padding:2px 8px; cursor:pointer; font-size:12px; color:#DC2626; }
    .storage-info { background:#F3F4F6; border-radius:8px; padding:10px;
      font-size:13px; color:#6B7280; margin-top:8px; }
  </style>
</head><body>
  <h2><img alt="notepad icon" data-bbox="78 763 94 780" style="vertical-align: middle;"/> Notas Persistentes</h2>
  <div class="panel">
    <textarea id="inp-nota" placeholder="Escribe tu nota aquí..."></textarea>
    <div class="botones">
      <button class="btn-guardar" onclick="gestor.guardar()"><img alt="save icon" data-bbox="723 873 739 890" style="vertical-align: middle;"/> Guardar</button>
      <button class="btn-limpiar" onclick="gestor.limpiarTodo()"><img alt="trash icon" data-bbox="763 903 780 920" style="vertical-align: middle;"/> Borrar
    </div>
  </div>
  <div class="storage-info">
    <p>Última actualización: <span></span></p>
    <p>Total de notas guardadas: <span></span></p>
  </div>
</body></html>
```

```

<div class="panel">
  <strong>📅 Mis notas (<span id="cont-notas">0</span>)</strong>
  <div id="lista-notas"></div>
  <div class="storage-info" id="storage-info"></div>
</div>
<script>/* código JS */</script>
</body></html>

```

JavaScript — Paso a paso

```

class GestorNotas {

  // Clave que usaremos en localStorage
  static CLAVE = "mis_notas_poo";

  constructor() {
    // Al crear la instancia, cargamos las notas guardadas
    this.notas = this.cargarDeStorage();
    this.render();
  }

  cargarDeStorage() {
    // localStorage.getItem devuelve null si no existe la clave
    const json = localStorage.getItem(GestorNotas.CLAVE);
    if (!json) return [];

    // JSON.parse convierte el STRING guardado de vuelta a array de objetos
    return JSON.parse(json);
  }

  guardarEnStorage() {
    // JSON.stringify convierte el array de objetos a un STRING
    // para que localStorage pueda almacenarlo
    const json = JSON.stringify(this.notas);
    localStorage.setItem(GestorNotas.CLAVE, json);
  }

  guardar() {
    const texto = document.getElementById("inp-nota").value.trim();
    if (!texto) return;

    const nota = {
      id:      Date.now(), // timestamp único como ID
      texto,
      fecha:   new Date().toLocaleString()
    };

    this.notas.unshift(nota); // al inicio del array
    this.guardarEnStorage();
    document.getElementById("inp-nota").value = "";
    this.render();
  }
}

```

```

    }

    eliminar(id) {
        // Filtramos: nos quedamos con todo EXCEPTO la nota con ese id
        this.notas = this.notas.filter(n => n.id !== id);
        this.guardarEnStorage();
        this.render();
    }

    limpiarTodo() {
        if (!confirm("¿Borrar todas las notas?")) return;
        this.notas = [];
        localStorage.removeItem(GestorNotas.CLAVE);
        this.render();
    }

    render() {
        document.getElementById("cont-notas").innerHTML = this.notas.length;
        const lista = document.getElementById("lista-notas");
        lista.innerHTML = "";

        for (let i = 0; i < this.notas.length; i++) {
            const n = this.notas[i];
            lista.innerHTML +=
                "<div class='nota-card'>" +
                "  <div class='nota-fecha'>" + n.fecha + "</div>" +
                "  <div class='nota-texto'>" + n.texto + "</div>" +
                "  <button class='btn-del-nota'",
                    onclick="gestor.eliminar(" + n.id + ")\">X</button>" +
                "</div>";
        }

        // Mostrar tamaño en localStorage
        const bytes = (localStorage.getItem(GestorNotas.CLAVE) || "").length;
        document.getElementById("storage-info").innerHTML =
            "📄 Guardado en localStorage: ~" + bytes + " bytes";
    }
}

// La instancia se crea con las notas ya cargadas del navegador
const gestor = new GestorNotas();

```



Mini-reto para ti

Agrega una propiedad "color" a cada nota y un selector de color en el formulario.

Agrega un método exportar() que cree un JSON descargable: usa un <a> con href="data:text/json,...".

Bonus: Implementa un método buscar(termino) que filtre las notas sin borrarlas del array, solo ocultando las que no coinciden.

09



Galería de Productos con Filtros

Ejercicio 09 de 10 · Clase con renderizado, filtros dinámicos y búsqueda

Concepto

Una clase que encapsula toda la lógica de UI: datos, estado de filtros activos y métodos de renderizado. El principio de responsabilidad única aplicado a un componente de interfaz.

Metáfora

Esta clase es como un catálogo de tienda con vendedor incluido: el vendedor conoce el inventario, recuerda qué filtros aplicaste y reorganiza el mostrador cuando cambias de criterio.

Código HTML completo

```
<!DOCTYPE html>
<html lang="es"><head>
  <meta charset="UTF-8"><title>Galería con Filtros</title>
  <style>
    body { font-family:sans-serif; max-width:700px; margin:24px auto;
      padding:20px; background:#FFF8F7; }
    .filtros { display:flex; gap:8px; flex-wrap:wrap; margin-bottom:16px; }
    .chip { padding:8px 16px; border:2px solid #FECACA; background:white;
      border-radius:999px; cursor:pointer; font-size:14px;
      transition:all .2s; }
    .chip.activo { background:#E74C3C; color:white; border-color:#E74C3C; }
    .busqueda { width:100%; padding:10px 14px; font-size:15px;
      border:2px solid #FECACA; border-radius:10px;
      box-sizing:border-box; margin-bottom:14px; }
    .grid { display:grid; grid-template-columns:repeat(auto-fill,minmax(160px,1fr));
    gap:14px; }
    .card { background:white; border-radius:12px; padding:14px; text-align:center;
      box-shadow:0 2px 10px rgba(0,0,0,.08); transition:transform .2s; }
    .card:hover { transform:translateY(-4px); }
    .card-emoji { font-size:40px; }
    .card-nombre { font-weight:bold; font-size:14px; color:#4A0E0E; margin:8px 0 4px; }
    .card-precio { color:#E74C3C; font-weight:bold; font-size:16px; }
    .card-cat { font-size:11px; background:#FECACA; color:#7D2200;
      padding:2px 8px; border-radius:999px; display:inline-block; margin-
top:6px; }
    .stats-bar { background:#FFF0EE; border-radius:8px; padding:10px 14px;
      font-size:14px; color:#922B21; margin-bottom:12px; }
  </style>
</head><body>
  <h2>&img alt="Shopping cart icon" Tienda Online</h2>
  <input class="busqueda" id="busqueda" type="text"
    placeholder="🔍 Buscar productos..." oninput="tienda.filtrar()">
  <div class="filtros" id="filtros"></div>
  <div class="stats-bar" id="stats-bar"></div>
  <div class="grid" id="grid-productos"></div>
  <script>/* código JS */</script>
</body></html>
```

```
class Producto {
  constructor(nombre, precio, categoria, emoji) {
    this.nombre = nombre;
    this.precio = precio;
    this.categoria = categoria;
    this.emoji = emoji;
  }
}

// =====
// Clase Tienda: encapsula datos + estado de filtros + UI
// =====
class Tienda {
  constructor(productos) {
    this.productos = productos;
    this.categoriaActiva = "Todos"; // estado del filtro
    this.construirFiltros();
    this.filtrar();
  }

  obtenerCategorias() {
    const cats = ["Todos"];
    for (let i = 0; i < this.productos.length; i++) {
      const cat = this.productos[i].categoria;
      if (!cats.includes(cat)) cats.push(cat);
    }
    return cats;
  }

  construirFiltros() {
    const cont = document.getElementById("filtros");
    const cats = this.obtenerCategorias();
    cont.innerHTML = "";
    for (let i = 0; i < cats.length; i++) {
      const cat = cats[i];
      const clase = cat === this.categoriaActiva ? "chip activo" : "chip";
      cont.innerHTML +=
        "<span class=\"" + clase + "\"",
        onclick=\"tienda.setCategoria(' + cat + "')\"> " + cat + "</span>";
    }
  }

  setCategoria(cat) {
    this.categoriaActiva = cat;
    this.construirFiltros();
    this.filtrar();
  }

  filtrar() {
    const busq = document.getElementById("busqueda").value.toLowerCase();
    let resultado = [];
```

```

    for (let i = 0; i < this.productos.length; i++) {
      const p = this.productos[i];
      const coincideCategoria = this.categoriaActiva === "Todos" ||
        p.categoria === this.categoriaActiva;
      const coincideBusqueda = p.nombre.toLowerCase().includes(busq);
      if (coincideCategoria && coincideBusqueda) resultado.push(p);
    }

    this.renderGrid(resultado);
  }

  renderGrid(lista) {
    // Calcular estadísticas del resultado
    let total = 0;
    for (let i = 0; i < lista.length; i++) total += lista[i].precio;
    const prom = lista.length ? (total / lista.length).toFixed(2) : 0;

    document.getElementById("stats-bar").innerHTML =
      lista.length + " productos · Precio promedio: $" + prom;

    const grid = document.getElementById("grid-productos");
    grid.innerHTML = "";
    for (let i = 0; i < lista.length; i++) {
      const p = lista[i];
      grid.innerHTML +=
        "<div class=\"card\">" +
        "  <div class=\"card-emoji\">" + p.emoji + "</div>" +
        "  <div class=\"card-nombre\">" + p.nombre + "</div>" +
        "  <div class=\"card-precio\">$" + p.precio.toFixed(2) + "</div>" +
        "  <span class=\"card-cat\">" + p.categoria + "</span>" +
        "</div>";
    }
  }
}

// Datos del catálogo
const tienda = new Tienda([
  new Producto("Auriculares", 59.99, "Electrónica", "🎧"),
  new Producto("Teclado RGB", 89.99, "Electrónica", "⌨️"),
  new Producto("Mochila", 34.50, "Accesorios", "🎒"),
  new Producto("Gorra", 19.90, "Accesorios", "🧢"),
  new Producto("Lámpara LED", 27.00, "Hogar", "💡"),
  new Producto("Planta Suculenta", 8.99, "Hogar", "🌵"),
  new Producto("Cuaderno A5", 12.50, "Papelería", "📓"),
  new Producto("Bolígrafos x10", 6.80, "Papelería", "🖋️"),
]);

```



Mini-reto para ti

Agrega un selector "Ordenar por: Precio ↑ / Precio ↓ / Nombre A-Z" que reordene los resultados.

Agrega una propiedad "favorito: false" a Producto. Agrega un botón  en cada tarjeta que la togglee y filtre por favoritos.

Bonus: Implementa un contador de "Productos vistos" que incremente cada vez que el usuario pasa el ratón sobre una tarjeta (evento mouseover).

10



Mini App: Banco Virtual — Proyecto Integrador

Ejercicio 10 de 10 · Herencia, encapsulamiento, localStorage, métodos estáticos

Concepto

Proyecto final que combina todos los conceptos: clase base CuentaBancaria, subclases CuentaAhorro y CuentaCorriente con comportamientos distintos, persistencia y estadísticas estáticas.

Metáfora

Es un banco real simplificado: el banco tiene reglas generales para todas las cuentas (clase base), pero cada tipo de cuenta (ahorro vs corriente) tiene sus propias reglas de interés y límites.



Código HTML completo

```
<!DOCTYPE html>
<html lang="es"><head>
  <meta charset="UTF-8"><title>Banco Virtual</title>
  <style>
    * { box-sizing:border-box; }
    body { font-family:sans-serif; max-width:680px; margin:24px auto;
      padding:20px; background:#E0F2F1; }
    h2 { color:#004D40; }
    .tabs { display:flex; gap:6px; margin-bottom:14px; }
    .tab { padding:10px 18px; background:white; border:none; border-radius:8px 8px 0 0;
      cursor:pointer; font-size:14px; color:#555; border-bottom:3px solid
transparent; }
    .tab.activa { color:#00897B; border-bottom-color:#00897B; font-weight:bold; }
    .panel { background:white; border-radius:0 14px 14px 14px;
      padding:20px; box-shadow:0 2px 12px rgba(0,0,0,.08); }
    .saldo-grande { font-size:46px; font-weight:bold; color:#00897B; text-align:center;
margin:8px 0; }
    .tipo-badge { text-align:center; margin-bottom:16px; }
    .badge { display:inline-block; padding:4px 12px; border-radius:999px;
      font-size:13px; font-weight:bold; }
    .badge-ahorro { background:#B2EBF2; color:#006064; }
    .badge-corriente{ background:#C8E6C9; color:#1B5E20; }
    .fila-op { display:flex; gap:8px; margin-bottom:10px; }
    input { flex:1; padding:10px; font-size:15px; border:2px solid #B2DFDB; border-
radius:8px; }
    .btn-dep { padding:10px 16px; background:#B2DFDB; color:#004D40;
      border:none; border-radius:8px; font-weight:bold; cursor:pointer; }
    .btn-ret { padding:10px 16px; background:#FFCDD2; color:#B71C1C;
      border:none; border-radius:8px; font-weight:bold; cursor:pointer; }
    .mov-list { max-height:180px; overflow-y:auto; margin-top:12px; }
    .mov { display:flex; justify-content:space-between; padding:8px 12px;
      border-radius:6px; margin-bottom:4px; font-size:14px; }
    .dep { background:#E0F2F1; color:#004D40; }
    .ret { background:#FFEBEE; color:#B71C1C; }
    .int { background:#FFF9C4; color:#F57F17; }
    .err-msg { color:#C62828; font-size:13px; text-align:center; margin-top:6px; }
```

```

    .stat-grid { display:grid; grid-template-columns:repeat(3,1fr); gap:10px; }
    .stat { background:#E0F2F1; border-radius:10px; padding:14px; text-align:center; }
    .stat .n { font-size:28px; font-weight:bold; color:#00695C; }
    .crear-form { display:grid; gap:8px; }
    .crear-form select { padding:10px; font-size:15px; border:2px solid #B2DFDB;
border-radius:8px; }
    #lista-cuentas { display:flex; gap:8px; flex-wrap:wrap; margin-bottom:14px; }
    .chip-cuenta { padding:8px 14px; border:2px solid #B2DFDB; background:white;
        border-radius:999px; cursor:pointer; font-size:14px; }
    .chip-cuenta.activo { background:#00897B; color:white; border-color:#00897B; }
</style>
</head><body>
<h2>&img alt="Bank icon" data-bbox="161 254 176 266"/> Banco Virtual POO</h2>
<div class="tabs">
    <button class="tab activa" onclick="mostrarTab('cuentas')">&img alt="Wallet icon" data-bbox="241 286 256 298"/> Cuentas</button>
    <button class="tab" onclick="mostrarTab('nueva')">&img alt="Plus icon" data-bbox="291 301 306 313"/> Nueva cuenta</button>
    <button class="tab" onclick="mostrarTab('stats')">&img alt="Bar chart icon" data-bbox="341 316 356 328"/> Estadísticas</button>
</div>
<div id="tab-cuentas" class="panel">
    <div id="lista-cuentas"></div>
    <div id="panel-cuenta"></div>
</div>
<div id="tab-nueva" class="panel" style="display:none">
    <h3>Nueva cuenta</h3>
    <div class="crear-form">
        <input id="nw-titular" type="text" placeholder="Nombre del titular">
        <input id="nw-saldo" type="number" placeholder="Saldo inicial">
        <select id="nw-tipo">
            <option value="ahorro">Cuenta de Ahorro (interés 2%)</option>
            <option value="corriente">Cuenta Corriente (sin interés)</option>
        </select>
        <button onclick="banco.crearCuenta()"
            style="padding:11px;background:#00897B;color:white;border:none;
            border-radius:8px;font-size:15px;cursor:pointer;">
            Crear cuenta
        </button>
    </div>
</div>
<div id="tab-stats" class="panel" style="display:none">
    <h3>Estadísticas del banco</h3>
    <div class="stat-grid" id="stat-grid"></div>
</div>
<script>/* código JS */</script>
</body></html>

```

JavaScript — Paso a paso

```

// =====
// CLASE BASE con encapsulamiento (#)
// =====
class CuentaBancaria {
    #saldo = 0;
    #movimientos = [];
    static totalCuentas = 0;

```

```

    constructor(titular, saldoInicial, tipo) {
        this.titular = titular;
        this.tipo     = tipo;
        this.id        = ++CuentaBancaria.totalCuentas;
        if (saldoInicial > 0) {
            this.#saldo = saldoInicial;
            this.#movimientos.push({ tipo:"apertura", monto:saldoInicial, fecha:new
Date().toLocaleTimeString() });
        }
    }

    get saldo() { return this.#saldo; }

    depositar(monto) {
        if (monto <= 0) throw new Error("Monto inválido");
        this.#saldo += monto;
        this.#movimientos.unshift({ tipo:"deposito", monto, fecha:new
Date().toLocaleTimeString() });
    }

    retirar(monto) {
        if (monto <= 0) throw new Error("Monto inválido");
        if (monto > this.#saldo) throw new Error("Saldo insuficiente");
        this.#saldo -= monto;
        this.#movimientos.unshift({ tipo:"retiro", monto, fecha:new
Date().toLocaleTimeString() });
    }

    getMovimientos() { return [...this.#movimientos].slice(0,8); }

    // Método que las subclases pueden override
    calcularInteres() { return 0; }

    toJSON() {
        return { titular:this.titular, saldo:this.#saldo,
                tipo:this.tipo, id:this.id,
                movimientos: this.#movimientos };
    }
}

// =====
// SUBCLASES - herencia y polimorfismo
// =====
class CuentaAhorro extends CuentaBancaria {
    constructor(titular, saldoInicial) {
        super(titular, saldoInicial, "ahorro");
        this.tasaInteres = 0.02; // 2% mensual
    }

    // Override: el ahorro SÍ genera interés
    calcularInteres() {
        return this.saldo * this.tasaInteres;
    }
}

```

```

    aplicarInteres() {
        const interes = this.calcularInteres();
        this.depositar(interres);
        return interes;
    }
}

class CuentaCorriente extends CuentaBancaria {
    constructor(titular, saldoInicial) {
        super(titular, saldoInicial, "corriente");
        this.sobregiro = 200; // puede quedar hasta -200
    }

    // Override: permite sobregiro
    retirar(monto) {
        if (monto <= 0) throw new Error("Monto inválido");
        if (monto > this.saldo + this.sobregiro)
            throw new Error("Excede límite de sobregiro ($" + this.sobregiro + ")");
        // Llamamos al retirar del padre solo si hay saldo suficiente
        if (monto <= this.saldo) {
            super.retirar(monto);
        } else {
            // Usamos directamente el depósito interno para simular sobregiro
            this.depositar(0 - monto); // truco: depositar negativo
        }
    }
}

// =====
// CLASE BANCO: gestiona el array de cuentas + persistencia
// =====
class Banco {
    constructor() {
        this.cuentas = [];
        this.seleccionada = 0;
    }

    crearCuenta() {
        const titular = document.getElementById("nw-titular").value.trim();
        const saldo = Number(document.getElementById("nw-saldo").value);
        const tipo = document.getElementById("nw-tipo").value;
        if (!titular) { alert("Escribe el nombre del titular"); return; }
        const cuenta = tipo === "ahorro"
            ? new CuentaAhorro(titular, saldo)
            : new CuentaCorriente(titular, saldo);
        this.cuentas.push(cuenta);
        this.seleccionada = this.cuentas.length - 1;
        mostrarTab("cuentas");
        this.renderTodo();
    }

    seleccionar(i) {
        this.seleccionada = i;
        this.renderTodo();
    }
}

```

```

depositar() {
    const monto = Number(document.getElementById("monto-op").value);
    const err    = document.getElementById("err-op");
    try { this.cuentas[this.seleccionada].depositar(monto); err.innerHTML="";
this.renderTodo(); }
    catch(e) { err.innerHTML = "⚠️ " + e.message; }
}

retirar() {
    const monto = Number(document.getElementById("monto-op").value);
    const err    = document.getElementById("err-op");
    try { this.cuentas[this.seleccionada].retirar(monto); err.innerHTML="";
this.renderTodo(); }
    catch(e) { err.innerHTML = "⚠️ " + e.message; }
}

aplicarIntereses() {
    for (let i = 0; i < this.cuentas.length; i++) {
        if (this.cuentas[i] instanceof CuentaAhorro) {
            this.cuentas[i].aplicarInteres();
        }
    }
    this.renderTodo();
}

renderTodo() {
    this.renderSelectorCuentas();
    if (this.cuentas.length > 0) this.renderPanelCuenta();
    this.renderStats();
}

renderSelectorCuentas() {
    const cont = document.getElementById("lista-cuentas");
    cont.innerHTML = "";
    for (let i = 0; i < this.cuentas.length; i++) {
        const c      = this.cuentas[i];
        const clase = i === this.seleccionada ? "chip-cuenta activo" : "chip-cuenta";
        cont.innerHTML +=
            "<span class=\"" + clase + "\" onclick=\"banco.seleccionar(" + i + ")\">" +
            c.titular + "</span>";
    }
}

renderPanelCuenta() {
    const c      = this.cuentas[this.seleccionada];
    const badge = c.tipo === "ahorro"
        ? "<span class=\"badge badge-ahorro\">💰 Ahorro · Interés " + (c.tasaInteres*100)
+ "%</span>"
        : "<span class=\"badge badge-corriente\">🏠 Corriente · Sobregiro $" +
c.sobregiro + "</span>";

    let movsHtml = "";
    const movs = c.getMovimientos();
    for (let i = 0; i < movs.length; i++) {
        const m = movs[i];

```




Mini-reto para ti

Agrega persistencia: cuando se crea o modifica una cuenta, guarda el estado en localStorage con JSON.stringify.

Crea una subclase CuentaJoven que extienda CuentaAhorro con una tasa de interés del 3.5% pero con límite de depósito de \$500 por operación.

Bonus final: Agrega un método estático Banco.crearDesdeJSON(json) que restaure las cuentas desde localStorage al recargar la página.



¡Lo lograste! — Resumen de los 10 conceptos POO

Concepto	Lo aprendiste en	Lo verás en React/Vue como
class + constructor	Ej. 01 — Mascota Virtual	Componentes con estado propio
Múltiples instancias	Ej. 02 — Múltiples Mascotas	Listas de componentes (map)
# Private + get/set	Ej. 03 — Cuenta Bancaria	Estado interno protegido
extends + super()	Ej. 04 — Figuras Geométricas	Higher-Order Components
Composición de clases	Ej. 05 — Carrito de Compras	Props anidadas / Context
this + bind / arrow fn	Ej. 06 — Eventos y Binding	Handlers en componentes
static methods	Ej. 07 — Estadísticas	Utility functions / helpers
localStorage + JSON	Ej. 08 — Notas Persistentes	Zustand / Redux persist
Clase-componente UI	Ej. 09 — Galería con Filtros	Custom hooks / stores
Proyecto integrador POO	Ej. 10 — Banco Virtual	Aplicación real completa



Tus siguientes pasos

- ◆ TypeScript: añade tipos estáticos a tus clases. Es POO con superpoderes.
- ◆ Patrón MVC: separa tu app en Modelo (clases de datos), Vista (DOM) y Controlador (eventos).
- ◆ React: cada componente funcional es una función con estado. Con lo que sabes de clases, los hooks son lógicos.
- ◆ Node.js + Express: usa clases para modelar entidades en el backend (User, Product, Order).
- ◆ Patrón Observador: implementa tu propio sistema de eventos con una clase EventEmitter.